MEDICAPS UNIVERSITY

File Handling in Php

Laxmikant Soni

Learning Objectives

- By the end of this lecture, students will:
  - Understand the concept and importance of file handling in PHP.
  - Learn how to open, read, write, and close files using PHP functions.
  - Apply functions such as `fopen()`, `fread()`, `fwrite()`, and `fclose()` in practical scenarios.
  - Handle file operations securely with error checking.
  - Create programs that perform common tasks like logging, reading configuration data, and storing user input in files.

PHP File I/O Functions

Function Name(s)	Category
`file`, `file_get_contents`, `file_put_contents`	Reading/Writing entire files
`basename`, `file_exists`, `filesize`, `fileperms`, `filemtime`, `is_dir`, `is_readable`, `is_writable`, `disk_free_space`	Asking for information
`copy`, `rename`, `unlink`, `chmod`, `chgrp`, `chown`, `mkdir`, `rmdir`	Manipulating files and directories
`glob`, `scandir`	Reading directories

Reading/Writing Files

Function	Description	Example Usage
`fopen()`	Opens a file in a specified mode	`fopen("data.txt", "r")`
`fread()`	Reads from an open file	`fread($handle, 1024)`
`fwrite()`	Writes data to an open file	`fwrite($handle, "Hello World")`
`fclose()`	Closes an open file	`fclose($handle)`
`file_get_contents()`	Reads entire file into a string	`$data = file_get_contents("data.txt")`
`file_put_contents()`	Writes a string to a file (creates/overwrites)	`file_put_contents("data.txt","Hi")`

File Modes in PHP

Mode	Description	Example
`r`	Open for reading only; file pointer starts at the beginning	`fopen("data.txt", "r")`
`r+`	Open for reading and writing; file pointer starts at the beginning	`fopen("data.txt", "r+")`
`w`	Open for writing only; erases contents of file or creates new	`fopen("data.txt", "w")`
`w+`	Open for reading and writing; truncates or creates new file	`fopen("data.txt", "w+")`
`a`	Open for writing only; file pointer at end; creates new if not exists	`fopen("data.txt", "a")`
`a+`	Open for reading and writing; file pointer at end; creates new if not exists	`fopen("data.txt", "a+")`
`x`	Create new file for writing only; fails if file already exists	`fopen("data.txt", "x")`
`x+`	Create new file for reading and writing; fails if file already exists	`fopen("data.txt", "x+")`

File Modes in PHP

Text vs Binary: Add `"b"` to the mode (e.g., `"rb"`, `"wb"`) when dealing with binary files.
Always close files after use with **`fclose()`** to free resources.

Difference: `file("abc.txt")` vs `file_get_contents("abc.txt")`

- **`file("abc.txt")`**
  - Reads the file into an **array**.
  - Each element of the array represents a **line** from the file.
  - Example: `print_r(file("abc.txt"));`
- **`file_get_contents("abc.txt")`**
  - Reads the entire file into a **single string**.
  - Useful when you want the complete file contents at once.
  - Example: `echo file_get_contents("abc.txt");`

Reading-writing an entire file

- Read the entire file into a string
- Reverse the string using `strrev()`
- Write the reversed content back to the file

```
<?php
$text = file_get_contents("poem.txt");   // read
$text = strrev($text);                   //
reverse
file_put_contents("poem.txt", $text);    //
write back
?>
```


Appending to a File in PHP

- Use `file_put_contents()` with the `FILE_APPEND` flag

- This ensures the new text is added at the end of the file, preserving existing content

```php
<?php
$new_text = "P.S. ILY, GTG TTYL!~";
file_put_contents("poem.txt", $new_text,
FILE_APPEND);
?>
```

The `file()` Function in PHP

The `file()` function in PHP reads the entire file into an array.
Each line of the file becomes an element of the array, including the newline character `\n`.

Syntax

```
array file ( string $filename [, int $flags = 0 [, resource $context ]] )
```

The `file()` Function Parameters

Parameter	Description
`$filename`	The name of the file to read.
`$flags` (optional)	Can be used to modify behavior, such as:• `FILE_IGNORE_NEW_LINES` → Removes newline characters.• `FILE_SKIP_EMPTY_LINES` → Skips empty lines.
`$context` (optional)	A context stream.

Difference between `fopen()` and `file()` in PHP

Feature	`fopen()`	`file()`
Purpose	Opens a file for reading/writing with a file pointer	Reads the entire file into an array
Return Value	Returns a file handle (resource)	Returns an array of lines
Usage	Used with functions like `fgets()`, `fwrite()`, `fclose()`	Directly processes file content line by line
Memory Usage	Efficient for large files (reads gradually)	Loads entire file into memory (may be heavy for big files)
Example	`$fp = fopen("abc.txt", "r");`	`$lines = file("abc.txt");`

Closing a File in PHP

- After opening a file with `fopen()`, it is important to **close the file** when you are done.

- This releases system resources and ensures that all data is written properly (especially in write/append modes).

- PHP provides the `fclose()` function for this purpose.

Example

```php
<?php
$fp = fopen("example.txt", "w");
fwrite($fp, "Hello, World!");
fclose($fp);  // closes the file
?>
```


Why We Don't Need to Close a File When Using `file()` in PHP

- The `file()` function in PHP is a **high-level function** that:
  - Opens the file.
  - Reads the entire content into an array (each line as an element).
  - Closes the file automatically.
- Because the **open–read–close cycle is handled internally**, the programmer does not need to explicitly call `fclose()`.

Example

```php
<?php
$lines = file("example.txt", FILE_IGNORE_NEW_LINES);
print_r($lines);
?>
```

- With fopen() → You must close using fclose().
- With file() → PHP closes it for you, so no manual closing is needed.

Reading a File

- **fread()**
  - Reads a file up to a specified length (in bytes).
  - Useful when you don’t want to load the entire file at once.

```php
<?php
$handle = fopen("sample.txt", "r");
$content = fread($handle,
filesize("sample.txt"));
fclose($handle);
echo $content;
?>
```

Reading a File - file_get_contents()

- **Reads the entire file into a single string.**

- **Simple and convenient when you need the whole content.**

```php
<?php
$data = file_get_contents("sample.txt");
echo $data;
?>
```

Difference between fread() and file_get_contents()

Feature	fread()	file_get_contents()
Purpose	Reads a file up to a specified length.	Reads the entire file into a single string.
Usage	Requires an open file handle (`fopen`).	Works directly with the filename.
Control	Gives more control (e.g., read part of a file, binary data).	Simpler, but always reads the full content.
When to Use	Useful for reading large files in chunks or partial content.	Convenient when the whole file is needed.
Closing File	Must manually close the file using `fclose()`.	No need to close file, handled automatically.

Example:

```php
// fread() example
$handle = fopen("sample.txt", "r");
$content = fread($handle,
filesize("sample.txt"));
fclose($handle);
echo $content;

// file_get_contents() example
$data = file_get_contents("sample.txt");
echo $data;
```

fwrite(): Writing to a File

- The **`fwrite()`** function in PHP is used to write data to a file.
- It requires a valid file handle, usually obtained using `fopen()`.
- If the file is opened in write (`w`) or append (`a`) mode, `fwrite()` will add data accordingly.
- After writing, it is good practice to close the file using `fclose()`.

Syntax:

```
fwrite(file_handle, string, length);
```

- file_handle → The file pointer obtained from fopen().
- string → The data you want to write.
- length (optional) → Maximum number of bytes to write. If omitted, it writes the entire string.

example - fwrite()

```php
<?php
$handle = fopen("example.txt", "w"); // Open file for
writing
fwrite($handle, "Hello, World!\n");  // Write data
fwrite($handle, "PHP file handling is powerful.\n");
fclose($handle);                     // Close file
echo "Data written successfully!";
?>
```

- If the file does not exist, it will be created.
- If opened in “w” mode, existing content will be erased before writing.
- If opened in “a” mode, data will be appended without erasing.

fwrite() vs file_put_contents()

Feature	fwrite()	file_put_contents()
Definition	Writes data to a file using a file handle obtained via `fopen()`.	Writes data to a file directly, without using `fopen()`.
File Handle	Requires an open file pointer (`fopen()`).	No file handle needed, handles opening/closing itself.
Simplicity	More manual: you must open, write, and close the file.	Simpler: one function call handles everything.
Modes	Supports multiple modes (`w`, `a`, `r+`, etc.).	By default overwrites file; can append with `FILE_APPEND` flag.
Performance	Efficient for writing large amounts of data gradually.	Better for quick writing of small/medium data in one go.
Error Handling	More control, since you manage `fopen()`, `fwrite()`, `fclose()`.	Less control, error handling is limited.
Use Case	Writing data in parts (e.g., logs, loops, streams).	Writing complete content at once (e.g., config, cache, reports).

Example using fwrite() and file_put_contents()

```php
<?php
$handle = fopen("example.txt", "a");
fwrite($handle, "Appending with fwrite()\n");
fclose($handle);
?>
```

```php
<?php
file_put_contents("example.txt", "Writing with file_put_contents()\n", FILE_APPEND);
?>
```

Copying a File

In PHP, you can copy a file from one location to another using the **`copy()`** function.

Example

```php
<?php
$source = "sample.txt";
$destination = "backup.txt";

if (copy($source, $destination)) {
    echo "File copied successfully!";
} else {
    echo "Failed to copy file.";
}
?>
```


Renaming

In PHP, the **`rename()`** function is used to **rename** a file or **move** it to a different location.

example

```php
<?php
$oldName = "sample.txt";
$newName = "document.txt";

if (rename($oldName, $newName)) {
    echo "File renamed successfully!";
} else {
    echo "Failed to rename file.";
}
?>
```